

南京理工大学紫金学院

《计算机综合实验》实验报告



课程编号:	03418011
课程名称:	计算机综合实验
学 院:	计算机与人工智能学院
专 业:	计算机科学与技术
学 号:	2315302125
姓 名:	崔子霖
起止时间:	11 周-14 周
指导教师:	丛玉华

2026 年 5 月

一、实验目的

本实验面向《计算机综合实验》课程的大作业要求，围绕 ROS 与 Gazebo 完成一个可运行、可展示、可复现的综合仿真系统。项目在原有山地小车 Gazebo 仿真的基础上，继续扩展城市街景区域和空中飞机/无人机巡航效果，形成“山地 + 城市 + 地面小车 + 空中飞行器”的融合仿真环境。

- 掌握 ROS package、launch、topic、service、参数和节点之间的组织关系。
- 掌握 Gazebo Classic 中 world、SDF、URDF/Xacro、model:// 路径和模型资源的使用方法。
- 完成一个地面机器人和一个空中模型在同一仿真世界中的协同展示。
- 通过实际运行截图、演示视频和实验报告，完整说明项目设计、实现、测试和问题解决过程。

二、实验题目

基于 ROS + Gazebo 的山地-城市融合场景多机器人协同仿真系统。

项目目标是将原来的“山地小车 Gazebo 仿真”升级为一个综合演示系统：地面包含山地地形、山路、城市道路、建筑、人行道、路灯、树木、路牌和过渡连接道路；地面小车能够在道路中运行；空中飞机/无人机能够在融合场景上方巡航；所有内容通过一个 launch 文件一键启动。

三、实验内容与系统功能

功能模块	实现内容	演示效果
山地场景	保留 mountain_terrain mesh 山体，叠加山路、护栏、岩石、旗帜、入口门架、观景平台、营地和植被。	Gazebo 中能看到具有坡面和山路特征的山地环境。
城市街景	使用 SDF 基础几何体搭建道路、十字路口、建筑群、人行道、路灯、树木、公交站、广告牌、施工区和停车位。	城市区域层次明显，适合答辩展示。
山地城市融合	通过 mountain_city_transition 模型构建泥土路、碎石层、沥青入	山地出口自然过渡到城市道路，而不是两个割裂场景。

	口、路肩、护栏和路牌。	
地面小车	使用 URDF/Xacro 建模，保留 /cmd_vel 控制链路，并通过 Gazebo 插件实现运动。	小车可在道路附近生成并运行。
空中巡航	使用 simple_airplane/simple_drone SDF 模型，通过 uav_patrol_demo.py 调用 /gazebo/set_model_state。	飞机/无人机在城市和山地上空按固定轨迹巡航。
一键启动	mountain_city_air_demo.launch 同时启动 Gazebo、world、小车、控制节点和飞行节点。	答辩时一条命令启动完整系统。

四、实验采用的技术

1. 硬件平台

- Mac 主机运行 VMware Fusion 虚拟机。
- Ubuntu 20.04.5 LTS 虚拟机作为 ROS/Gazebo 实验环境。
- 仿真对象包括虚拟四轮小车、山地/城市场景模型、固定翼飞机和四旋翼无人机。

2. 软件开发环境

软件/工具	作用
ROS Noetic	负责节点、launch、topic、service、参数和 package 管理。
Gazebo Classic 11	负责三维场景、模型加载、物理仿真和可视化显示。
catkin_make	编译 ROS 工作空间和安装 Python 节点脚本。
Python 3 / rospy	编写小车控制、飞行巡航和模型重置节点。
SDF / URDF / Xacro	SDF 管理 Gazebo 模型和 world，URDF/Xacro 管理小车结构。
gnome-screenshot	Ubuntu 自带截图工具，用于采集真实运行截图。

3. 软件开发语言

- XML：编写 launch、SDF、URDF/Xacro 文件。
- Python：编写 ROS 控制节点，如 uav_patrol_demo.py、road_follow_car.py、reset_spawn_model.py。
- Markdown：整理 README、技术文档、答辩稿。

五、系统设计及实现

1. 总体设计

系统采用“一个综合 Gazebo world + 多个独立 SDF 模型 + 一个 URDF/Xacro 小车 + 多个 ROS 控制节点”的结构。world 文件负责把山地、城市、过渡区、飞机和无人机放入同一个仿真世界；launch 文件负责统一启动 Gazebo、生成小车并运行控制节点；Python 节点负责地面小车演示运行和空中巡航动画。

roslaunch mountain_car_sim mountain_city_air_demo.launch

层次	核心文件	说明
启动层	launch/ mountain_city_air_demo.launch	一键启动 Gazebo、综合 world、小车、飞行控制节点和道路跟随节点。
场景层	worlds/ mountain_city_air_demo.world	通过 model:// 引用山地、城市、过渡区和空中模型。
模型层	models/*/model.sdf	用 SDF/mesh 管理山地、城市、路灯、树木、建筑、飞机和无人机。
机器人层	urdf/mountain_car.xacro	定义四轮小车底盘、轮子、碰撞体、惯量和 Gazebo 控制插件。
控制层	scripts/uav_patrol_demo.py 等	通过 /cmd_vel 或 /gazebo/set_model_state 控制模型运动。

2. 山地场景实现

山地主体由 mountain_terrain 模型实现，核心文件位于 src/mountain_car_sim/models/mountain_terrain/model.sdf。该模型使用 DAE mesh 文件构造起

伏山体和山路，其中 `mountain_terrain.dae` 表示山体主体，`mountain_trail.dae` 表示贴合山地的蛇形道路。山地 `mesh` 同时作为 `visual` 和 `collision`，因此小车不是只显示在平面上，而是可以与山地碰撞体产生接触。

为了让山地更适合课程答辩展示，项目又新增 `mountain_showcase_details` 模型，在不破坏原有山地 `mesh` 的前提下叠加木护栏、入口门架、警示牌、城市方向牌、观景平台、帐篷营地、补给箱、营火、松树、灌木、碎石和风向袋等细节。这样山地不再只是一个裸地形，而是具有道路环境和展示层次的山地小车场景。

3. 城市场景实现

城市场景采用轻量 `SDF` 几何体搭建，重点是稳定可运行和视觉效果清楚。`simple_city/model.sdf` 中使用 `box` 搭建深灰色道路、支路和十字路口，使用浅灰色薄 `box` 搭建人行道，使用不同高度、不同颜色的 `box` 搭建建筑群，使用 `cylinder` 和 `sphere` 搭建路灯、树木和路牌。

为了使城市区域更像一个完整街区，又新增 `showcase_city_details` 模型，加入店铺门面、广告牌、交通龙门架、停车位、公交站、施工围挡、路面标线和飞行标识等元素。城市部分没有引入 `CARLA`、`OpenStreetMap` 或复杂 `city generator`，原因是课程展示更需要轻量、稳定、可复现，而不是依赖复杂外部环境。

4. 山地与城市连接设计

山地和城市没有做成两个分离 `world`，而是在 `mountain_city_air_demo.world` 中同时 `include` 山地、城市和过渡区模型。过渡区由 `mountain_city_transition/model.sdf` 实现，从山地出口到城市主路依次布置泥土路、碎石过渡层、沥青入口、路肩、路缘石、护栏、网关标识和绿化。这样小车从山地区域进入城市区域时，视觉上有“土路逐渐变为城市道路”的连续关系。

5. 小车模型与地面运动

小车模型保留原有 `mountain_car.xacro`。`Xacro` 文件定义 `base_link`、四个车轮、轮轴、碰撞体、惯量和颜色，并通过 `Gazebo` 插件接收 `/cmd_vel` 速度指令。最终 `launch` 中使用 `reset_spawn_model.py` 先删除旧模型再生成新模型，避免重复运行 `launch` 时出现 `entity already exists` 的问题。

地面运动方面，项目保留原有手动控制方式，同时新增 `road_follow_car.py` 用于答辩演示，让小车可以沿道路进行稳定展示运行。

6. 空中飞机/无人机巡航

空中模型使用 simple_airplane 和 simple_drone 两个轻量 SDF 模型。固定翼飞机由机身、机翼、尾翼、机头和颜色块组成，四旋翼无人机由中心机身、四个旋翼臂和旋翼圆柱组成。它们都不依赖 PX4、MAVROS 或复杂飞控环境。

飞行控制由 scripts/uav_patrol_demo.py 实现。该节点启动后等待 /gazebo/set_model_state 服务可用，然后根据时间计算巡航轨迹，持续设置模型 pose 和 twist，使 patrol_airplane 在固定高度附近沿椭圆轨迹巡航。yaw 根据运动方向变化，pitch 和 roll 保持简单稳定。选择这种方法的原因是课程项目更重视稳定展示，避免复杂飞控依赖导致答辩现场启动失败。

六、核心代码与文件说明

文件	作用
worlds/mountain_city_air_demo.world	综合 world，融合山地、城市、过渡道路、飞机和无人机模型。
launch/mountain_city_air_demo.launch	最终一键启动文件，加载 world、小车、飞行脚本和道路跟随脚本。
models/mountain_terrain/model.sdf	山地主体地形，使用 mesh 构建起伏山体和山路。
models/simple_city/model.sdf	城市主体，包括道路、建筑、人行道、树木、路灯等。
models/mountain_city_transition/model.sdf	山地到城市的融合连接道路。
models/simple_airplane/model.sdf	简化固定翼飞机模型。
urdf/mountain_car.xacro	四轮小车 URDF/Xacro 模型。
scripts/uav_patrol_demo.py	调用 /gazebo/set_model_state 控制飞机巡航。
scripts/road_follow_car.py	控制地面小车沿道路展示运行。

七、系统编译与运行

1. 编译命令

```
cd ~/catkin_ws
catkin_make -DCATKIN_WHITELIST_PACKAGES="mountain_car_sim"
source devel/setup.bash
```

由于虚拟机工作空间中可能存在其他实验包，为保证大作业项目编译稳定，本次实际运行时使用

CATKIN_WHITELIST_PACKAGES 只编译 mountain_car_sim。实际编译日志显示该包配置和生成成功，Python 脚本已安装到 devel/lib/mountain_car_sim。

2. 启动命令

```
roslaunch mountain_car_sim mountain_city_air_demo.launch
```

启动后 Gazebo 会加载 mountain_city_air_demo.world，并自动生成小车、飞机/无人机模型，随后启动小车道路跟随节点和空中巡航节点。

八、功能测试与实际运行截图

1. Gazebo 综合场景运行截图

下面截图使用 Ubuntu 自带 gnome-screenshot 工具采集，画面来自真实运行中的 Gazebo。截图中可以看到城市街景、道路、建筑、树木、路灯、空中飞机等元素，说明综合 world 已经成功加载并运行。

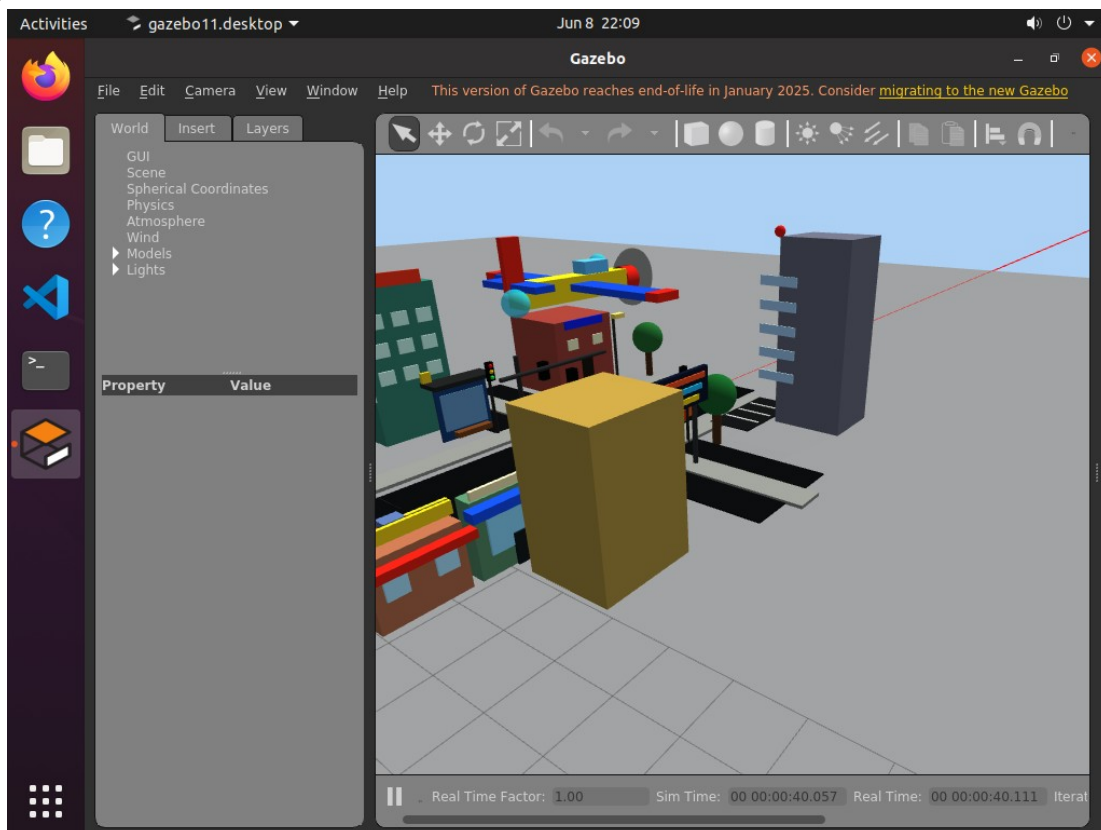


图 1 Gazebo 中运行的山地-城市融合仿真系统局部视图 (Ubuntu gnome-screenshot 实际截图)

2. ROS/Gazebo 运行状态截图

下图为 Ubuntu 终端中的实际运行状态截图，可以看到 /gazebo/model_states、/gazebo/set_model_state、/cmd_vel、/joint_states 等话题或服务，以及 roslaunch、gzserver、gzclient、road_follow_car.py、uav_patrol_demo.py 等进程。

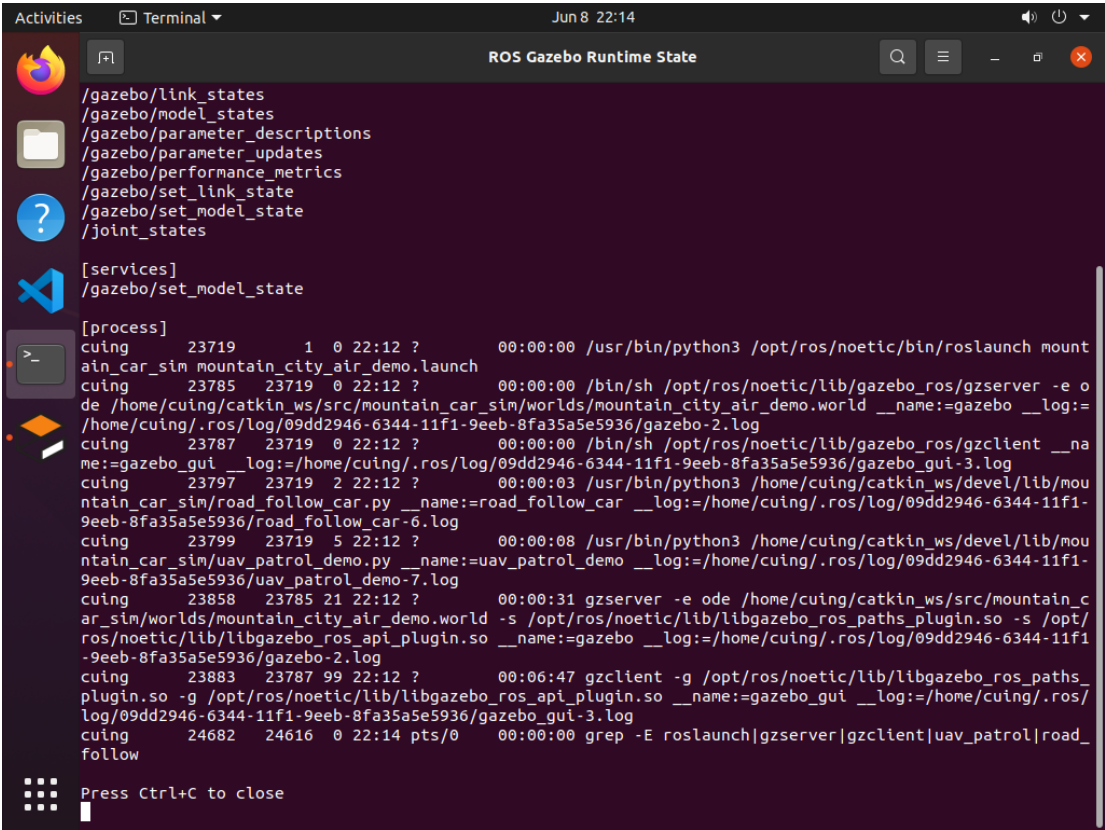


图 2 ROS/Gazebo 运行状态与关键话题服务截图 (Ubuntu gnome-screenshot 实际截图)

3. 功能测试结果

测试项	测试方法	结果
一键启动	执行 roslaunch mountain_car_sim mountain_city_air_demo.launch。	Gazebo、world、小车、飞行控制节点均启动。
城市场景显示	观察 Gazebo 画面中的道路、建筑、人行道、路灯、树木和路牌。	城市街景元素可见。
空中巡航	查看 Gazebo 画面和	飞机/无人机模型可见，巡航节点处

	/gazebo/set_model_state 服务。	于运行状态。
地面小车运行	查看 /cmd_vel 话题和 road_follow_car.py 进程。	地面控制链路存在，小车控制节点正常运行。
运行复现	在 Ubuntu VM 中重新同步源码、编译并启动。	项目可重新启动并采集真实截图。

九、遇到的问题及解决方法

1. Gazebo 模型依赖过重的问题

最初设想可以引用复杂城市模型或真实无人机飞控模型，但这类资源往往依赖重，可能需要 PX4、MAVROS、复杂 mesh 或外部资源下载。为保证课程答辩稳定，本项目最终采用 SDF 基础几何体手写城市街景，并使用简化飞机/无人机模型。这样虽然真实度不如专业仿真平台，但更稳定、更容易复现，也更适合课程展示。

2. 山地与城市割裂的问题

简单把城市放在山地旁边会显得两个场景割裂。解决方法是新增 mountain_city_transition 模型，在山地和城市之间加入泥土路、碎石层、沥青入口、路肩、路缘石、护栏、网关和绿化，使视觉上形成连续过渡。

3. 空中模型真实飞控复杂的问题

真实飞行控制需要空气动力学、飞控系统、传感器和控制器，课程项目中容易导致启动失败。本项目采用 /gazebo/set_model_state 直接设置模型状态，使飞机稳定按轨迹飞行。该方法适合演示级仿真，重点体现 ROS 节点和 Gazebo 服务之间的协作。

4. 重复运行 launch 时模型重名的问题

Gazebo 中同名模型重复 spawn 会出现 entity already exists。项目使用 reset_spawn_model.py，在生成 mountain_car 前先调用 /gazebo/delete_model 删除旧模型，再调用 /gazebo/spawn_urdf_model 生成新模型，提高重复演示的稳定性。

5. 虚拟机中截图和录屏的问题

报告截图要求使用 Ubuntu 自带截图功能，因此实际截图使用 `gnome-screenshot` 完成。功能演示视频则通过屏幕录制保存真实运行中的 Ubuntu/Gazebo 窗口画面，并与报告、PPT、源码一起放入最终提交文件夹。

十、实验总结

本项目从原始山地小车仿真出发，完成了山地、城市、地面小车、空中飞机/无人机的综合扩展。项目体现了 ROS 与 Gazebo 在机器人建模、场景搭建、节点控制、服务调用、多模型协同仿真方面的基本应用。

通过本次实验，我进一步理解了 ROS 中 `launch`、`topic`、`service`、`param`、`package` 的配合方式，也掌握了 Gazebo world、SDF 模型、URDF/Xacro 小车模型和 Python 控制节点的组织方法。项目最终能够一键启动、可截图、可录制演示视频，满足课程大作业的综合展示要求。

十一、答辩展示流程

- 第一步：打开 Ubuntu 终端，进入 `catkin_ws`，执行编译和 `source` 命令。
- 第二步：运行 `roslaunch mountain_car_sim mountain_city_air_demo.launch`。
- 第三步：在 Gazebo 中展示城市街景、道路、建筑、路灯、树木和飞机巡航。
- 第四步：说明山地、城市和过渡道路分别由哪些 SDF/mesh 模型实现。
- 第五步：说明小车使用 `/cmd_vel`，飞机使用 `/gazebo/set_model_state`。
- 第六步：总结本项目重点是多场景、多模型、多运动对象的综合仿真，而不是单一模型展示。

十二、PPT 与交付物说明

本次最终交付文件夹包含实验报告、源代码、功能演示视频和 PPT。PPT 共 6 页，内容包括项目概览、系统功能、实现技术架构、山地与城市建模方式、车辆运动与空中巡航实现、启动展示与总结。

[illegible][illegible][illegible][illegible][illegible]

图3 答辩 PPT 缩略预览